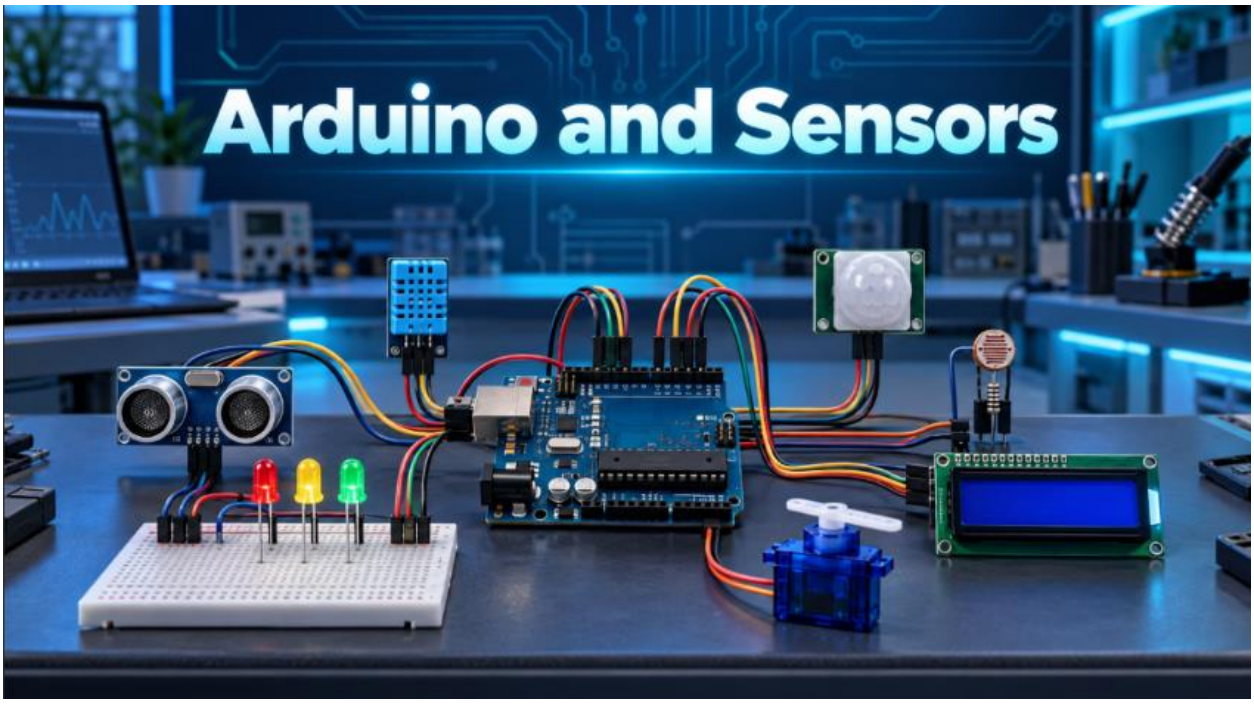
Arduino and Sensors Manual

Explanations, mini tasks and final project



Level Beginner to intermediate
Format Seven practical sessions of approximately two hours each
Made by Mostafa Amr

Workshop Overview

This manual introduces Arduino-based embedded systems through guided laboratory work. Students first learn how a microcontroller reads inputs and controls outputs, then connect light, temperature,

motion, and distance sensors. The workshop finishes with an integrated smart-room prototype. The emphasis is on understanding electrical connections, interpreting sensor data, writing readable code, testing systematically, and explaining engineering decisions.

Prerequisites and safety

- Basic familiarity with computers; prior programming experience is helpful but not required.
- Disconnect USB power before changing circuit wiring.
- Always use a current-limiting resistor with an LED.
- Check component polarity and pin labels before applying power.
- Use a common ground for every device in the circuit.
- Do not power motors or other high-current loads directly from an Arduino I/O pin.

Core equipment

Item	Typical quantity	Purpose
Arduino Uno and USB cable	1 per team	Controller and programming interface
Breadboard and jumper wires	1 set	Rapid circuit prototyping
LEDs and 220 ohm resistors	3 to 5	Visual outputs
Push buttons and 10 kohm resistors	2	Digital inputs
Potentiometer 10 kohm	1	Variable analog input
LDR and 10 kohm resistor	1 each	Light sensing voltage divider
DHT11 or DHT22	1	Temperature and humidity
HC-SR04	1	Ultrasonic distance
PIR module	1	Motion detection
Servo motor	1	Position actuator
Active or passive buzzer	1	Audio warning
I2C 16 by 2 LCD	1	User display

Assessment approach

Students should be assessed on working safely, explaining the circuit, producing readable code, testing edge cases, and interpreting results. A working prototype alone is not enough; students should be able to justify thresholds, identify sources of measurement error, and describe how the design could be improved.

Session 1 Arduino Fundamentals

Learning outcomes

- Identify the main parts of an Arduino Uno.

- Explain the difference between a microcontroller and a complete computer.
- Describe setup and loop.
- Configure and control a digital output.
- Upload, test, and debug a simple sketch.

Technical explanation

What Arduino is

Arduino is an open-source prototyping platform built around a microcontroller board and a programming environment. The Uno uses an ATmega328P microcontroller. Unlike a laptop, it normally runs one uploaded program repeatedly and interacts directly with electrical inputs and outputs.

Board architecture

The USB connector provides communication and can provide 5 V power. The voltage regulator supports an appropriate external supply. Digital pins represent two logic states, while analog input pins measure a range of voltages. The 5 V, 3.3 V, and GND pins supply reference power. The reset button restarts the current program without deleting it.

Sketch structure

setup runs once after power-up or reset and is used for initialization, such as selecting pin modes or starting serial communication. loop runs repeatedly for as long as the board has power. Variables store values, functions group actions, and comments document intent.

Digital output and current

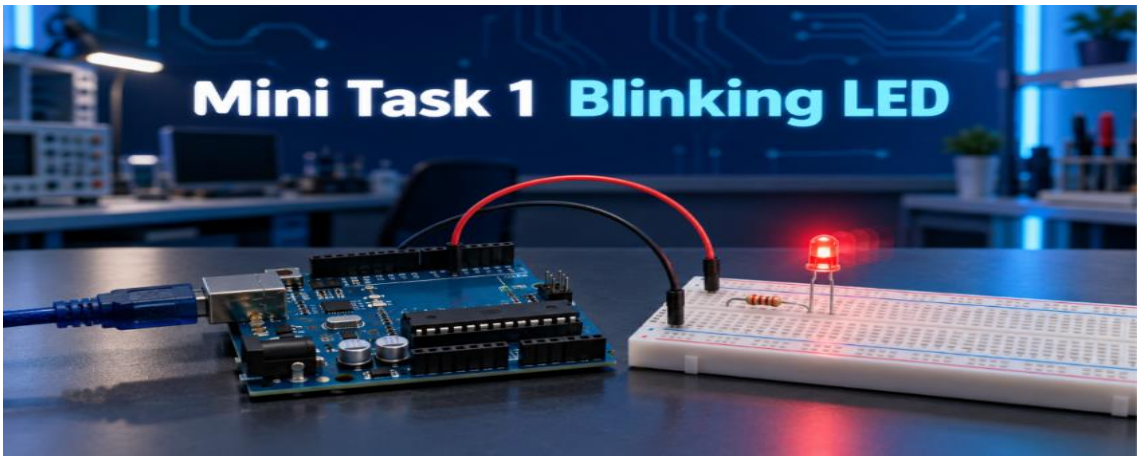
An output pin can be set HIGH or LOW. On a 5 V Uno, HIGH is approximately 5 V and LOW is approximately 0 V. An LED must be connected through a resistor. For a typical LED with about 2 V forward drop and a desired current near 10 to 15 mA, 220 ohm is a common classroom value.

Upload and debugging

The IDE compiles the sketch, checks syntax, and transfers machine code through USB. If uploading fails, verify the board model, selected port, USB cable, and whether another program is using the serial port. If the circuit does not behave as expected, test power, ground, polarity, wiring, and code separately.

Mini Task 1 Blinking LED

Objective. Control a digital output with timing.



Components

- Arduino Uno
- Built-in LED or external LED
- 220 ohm resistor if using an external LED

Connections

- For the built-in LED, use LED_BUILTIN.
- For an external LED, connect pin 8 to a 220 ohm resistor, then to the LED anode; connect the cathode to GND.

Procedure

1. Create a new sketch.
2. Configure the LED pin as OUTPUT in setup.
3. Set the pin HIGH, wait, set it LOW, and wait again.
4. Upload the sketch and change both delay values.

Expected result. The LED flashes continuously. Different ON and OFF delays produce different duty cycles.

Extension challenge. Replace delay-based timing with millis so the program can perform another task while the LED blinks.

```
const int LED_PIN = 8;


void setup() {
   pinMode(LED_PIN, OUTPUT);
 }


void loop() {
   digitalWrite(LED_PIN, HIGH);
   delay(1000);
   digitalWrite(LED_PIN, LOW);
   delay(1000);
 }
```

Mini Task 2 Traffic Light

Objective. Implement a predictable sequence using three digital outputs.



Components

- Red, yellow, and green LEDs
- Three 220 ohm resistors

Connections

- Red LED to pin 8, yellow to pin 9, and green to pin 10 through individual resistors.
- Connect all LED cathodes to GND.

Procedure

5. Configure all three pins as outputs.
6. Show red for five seconds.
7. Show red and yellow together for one second.
8. Show green for five seconds.
9. Show yellow for two seconds, then repeat.

Expected result. Only the intended traffic-light state is visible at each stage.

Extension challenge. Add a pedestrian request button and process the request safely at the end of the current green phase.

Session 2 Digital Inputs and Outputs

Learning outcomes

- Read a push button reliably.
- Explain floating inputs and pull resistors.
- Apply edge detection and basic debouncing.
- Build a toggle control and a reaction timer.

Technical explanation

Input voltage and logic

A digital input classifies voltage as LOW or HIGH. An unconnected input is floating and may change because of electrical noise. A pull-up or pull-down resistor gives the pin a known idle state.

Internal pull-up mode

INPUT_PULLUP connects a weak resistor internally between the input and 5 V. The button is then connected between the pin and ground. The logic is active-low: an unpressed button reads HIGH and a pressed button reads LOW.

Debouncing

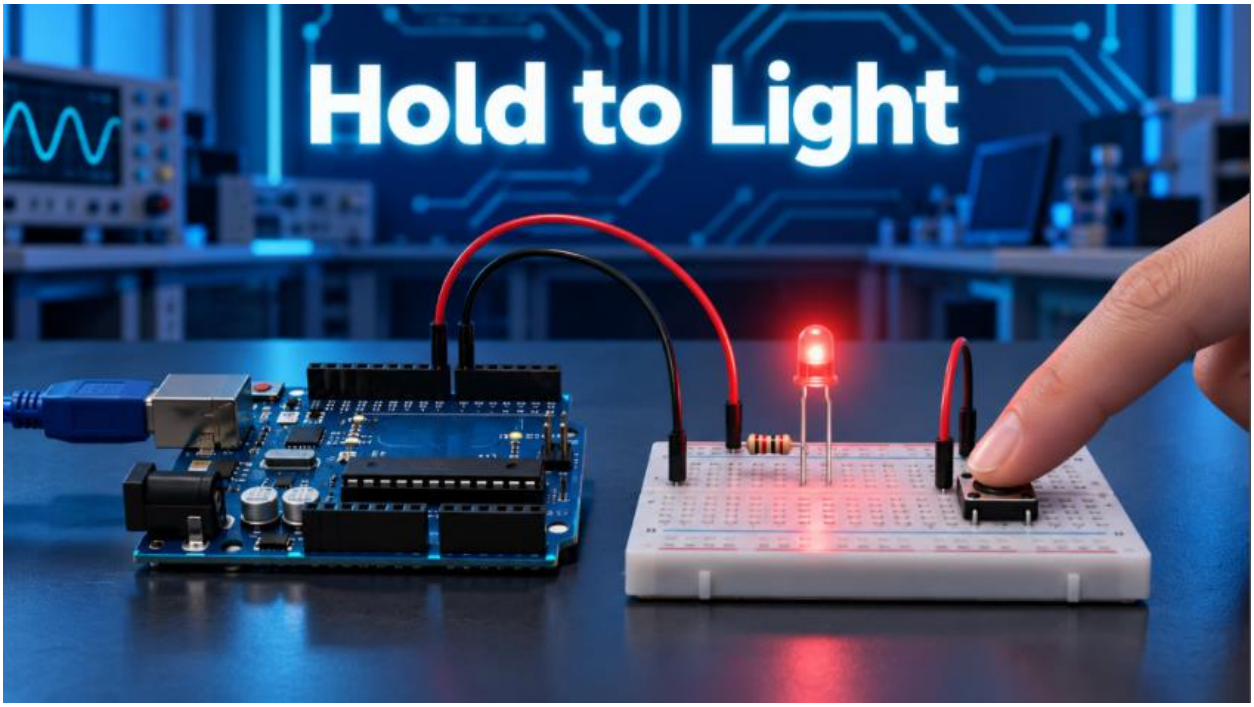
Mechanical contacts can make and break several times during one physical press. A program that toggles immediately on every reading may register multiple presses. Simple software debouncing waits for a stable state; more responsive designs compare timestamps using millis.

State and edge detection

A state describes the current condition, such as LED on or off. Edge detection compares the current button reading with the previous reading. A falling edge in an active-low circuit indicates a new press. This distinction prevents a held button from toggling the LED repeatedly.

Mini Task 1 Hold to Light

Objective. Make an LED follow a button state.



Components

- Push button
- LED and 220 ohm resistor

Connections

- Button between pin 2 and GND using INPUT_PULLUP.
- LED on pin 8 through 220 ohm to GND.

Procedure

10. Read the button.
11. Because the circuit is active-low, treat LOW as pressed.
12. Write the corresponding state to the LED.

Expected result. The LED is on only while the button is pressed.

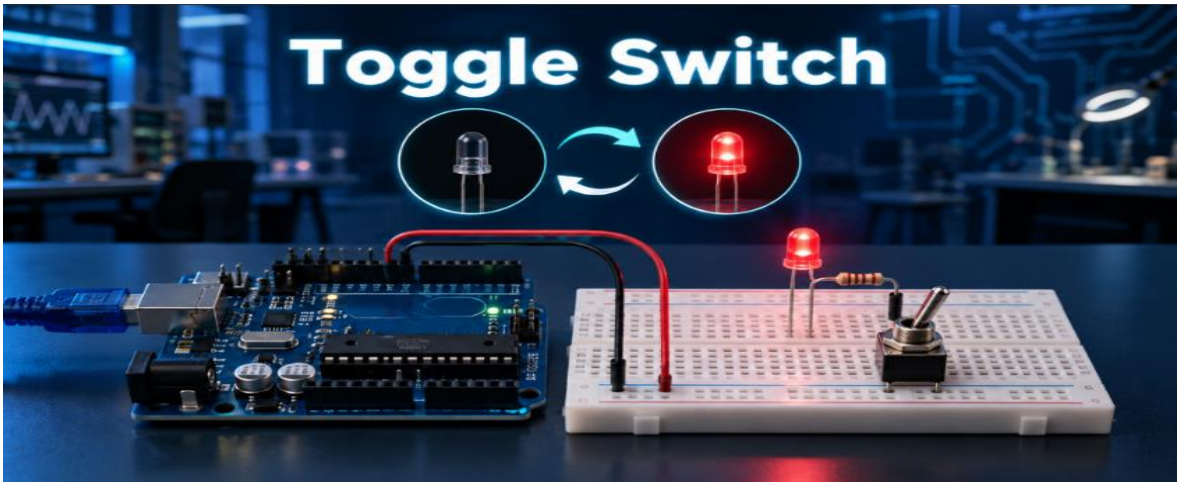
```
const int BUTTON_PIN = 2;
 const int LED_PIN = 8;


void setup() {
   pinMode(BUTTON_PIN, INPUT_PULLUP);
   pinMode(LED_PIN, OUTPUT);
 }


void loop() {
   bool pressed = digitalRead(BUTTON_PIN) == LOW;
   digitalWrite(LED_PIN, pressed ? HIGH : LOW);
 }
```

Mini Task 2 Toggle Switch

Objective. Change the LED state once per button press.



Components

- Same circuit as Mini Task 1

Connections

- No wiring changes are required.

Procedure

13. Store the previous button state and the LED state.
14. Detect a HIGH-to-LOW transition.
15. If at least 40 ms has passed since the last accepted press, invert the LED state.
16. Update the output and save the current reading.

Expected result. Each deliberate press changes the LED once.

Extension challenge. Create a reaction-time game: wait for a random interval, light the LED, and report the time until the user presses the button.

```
const int BUTTON_PIN = 2, LED_PIN = 8;
 bool ledState = false;
 bool previous = HIGH;
 unsigned long lastPress = 0;


void setup() {
   pinMode(BUTTON_PIN, INPUT_PULLUP);
   pinMode(LED_PIN, OUTPUT);
 }


void loop() {
   bool current = digitalRead(BUTTON_PIN);
   if (previous == HIGH && current == LOW && millis() - lastPress > 40) {
     ledState = !ledState;
     digitalWrite(LED_PIN, ledState);
     lastPress = millis();
   }
```

```
  previous = current;
}
```

Session 3 Analog Signals and Sensor Readings

Learning outcomes

- Explain analog-to-digital conversion.
- Build and analyze a voltage divider.
- Read and scale analog data.
- Use PWM to control apparent LED brightness.
- Select a threshold using measured data.

Technical explanation

Analog measurement

The Uno analog-to-digital converter represents an input from approximately 0 to 5 V as an integer from 0 to 1023. The approximate input voltage is reading multiplied by 5.0 divided by 1023. The reference and sensor noise limit real accuracy.

Voltage divider

Many resistive sensors must form a voltage divider. With a sensor resistance and a fixed resistor in series between 5 V and ground, the midpoint voltage changes as the sensor resistance changes. Swapping their positions reverses whether the reading increases or decreases.

PWM output

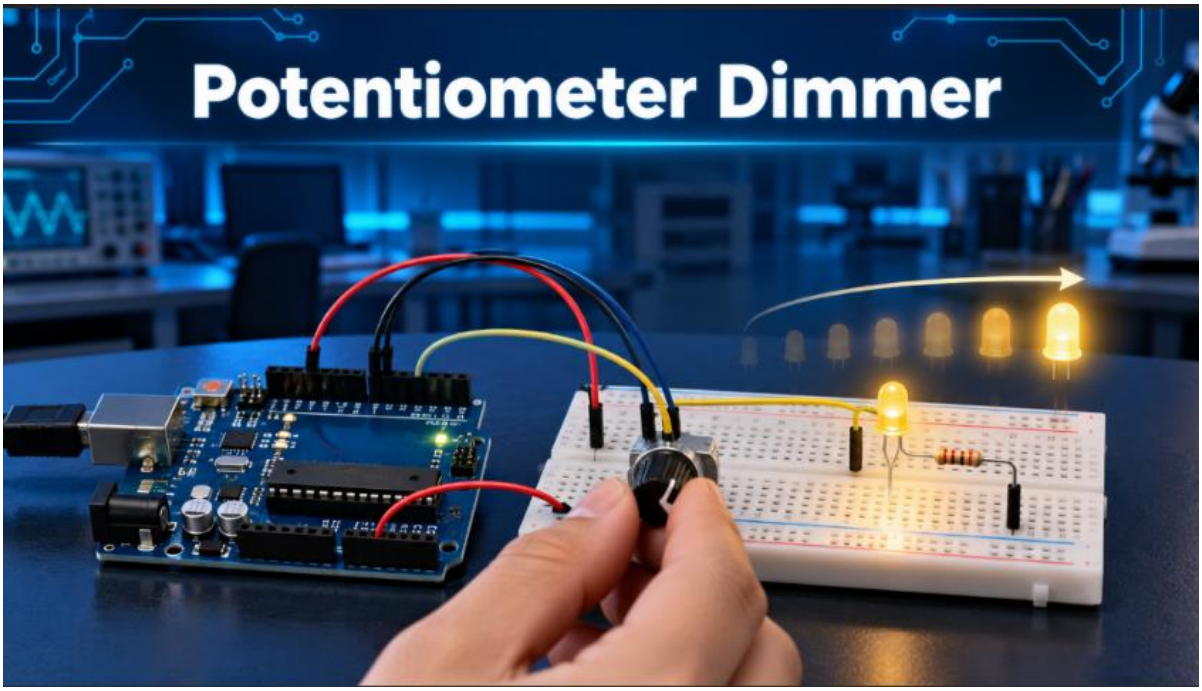
The Uno does not produce a true variable analog voltage on its normal I/O pins. analogWrite on a PWM pin rapidly switches the output. A value from 0 to 255 sets the duty cycle, changing average power and perceived LED brightness.

Calibration and thresholds

A fixed threshold copied from another circuit may fail because light, component tolerance, and sensor orientation differ. Record readings in bright and dark conditions, choose a value between the observed ranges, and add hysteresis if the output flickers near the threshold.

Mini Task 1 Potentiometer Dimmer

Objective. Map an analog input to a PWM output.



Components

- 10 kohm potentiometer
- LED and 220 ohm resistor

Connections

- Potentiometer outer pins to 5 V and GND; wiper to A0.
- PWM pin 9 to resistor, LED, then GND.

Procedure

17. Read A0.
18. Map 0 to 1023 into 0 to 255.
19. Send the mapped value to pin 9 with analogWrite.
20. Print both values for verification.

Expected result. Rotating the potentiometer changes LED brightness smoothly.

```
const int POT_PIN = A0, LED_PIN = 9;


void setup() { Serial.begin(9600); }


void loop() {
   int raw = analogRead(POT_PIN);
   int pwm = map(raw, 0, 1023, 0, 255);
```

```
  analogWrite(LED_PIN, pwm);
  Serial.print(raw); Serial.print(" -> "); Serial.println(pwm);
  delay(20);
}
```

Mini Task 2 Automatic Night Light

Objective. Use measured light data to make a control decision.



Components

- LDR
- 10 kohm resistor
- LED and 220 ohm resistor

Connections

- Connect 5 V to the LDR, the LDR to A0, and a 10 kohm resistor from A0 to GND.
- Connect the LED to pin 9 through 220 ohm, then GND.

Procedure

21. Print readings under bright, normal, and dark conditions.
22. Choose a threshold from the observed data.
23. Turn the LED on when the reading indicates darkness.
24. Test near the threshold and note any flicker.

Expected result. The LED turns on automatically in darkness.

Extension challenge. Add two thresholds, for example ON below 350 and OFF above 430, to create hysteresis and prevent flicker.

Session 4 Temperature and Humidity Sensors

Learning outcomes

- Describe a digital environmental sensor.
- Install and use a sensor library.
- Validate readings before using them.
- Implement a multi-level warning system.

Technical explanation

DHT sensor operation

A DHT11 or DHT22 contains a sensing element and internal electronics that transmit a calibrated digital data frame. DHT22 generally offers a wider range and better resolution. Modules may already include a pull-up resistor; a bare sensor normally needs one on the data line.

Libraries and abstraction

A library hides protocol timing and exposes functions such as readTemperature and readHumidity. Students should still understand pin assignments, sampling limits, return values, and the need to verify that readings are valid.

Sampling and validation

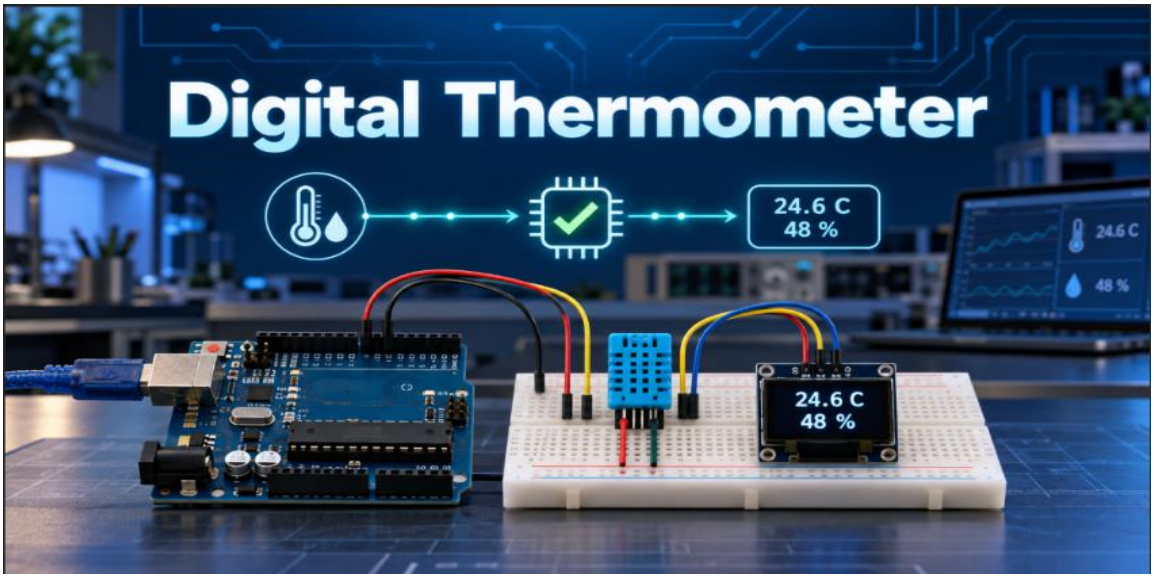
DHT sensors update slowly. Reading too frequently may return repeated or invalid values. A two-second interval is suitable for introductory work. The isnan check prevents invalid data from driving alarms.

Threshold engineering

A warning threshold must be defined for the application rather than selected arbitrarily. A classroom demonstration may use a convenient threshold, but a real system requires environmental requirements, sensor accuracy, placement, and hysteresis to be considered.

Mini Task 1 Digital Thermometer

Objective. Acquire and validate environmental data.



Components

- DHT11 or DHT22 module

Connections

- VCC to 5 V, GND to GND, DATA to pin 4.
- For a bare sensor, add the manufacturer-recommended pull-up between DATA and VCC.

Procedure

25. Install a compatible DHT library and its required dependency.
26. Select the correct sensor type in code.
27. Read every two seconds.
28. Check for invalid values before printing.

Expected result. Temperature and relative humidity appear in the Serial Monitor.

```
#include <DHT.h>
 #define DHT_PIN 4
 #define DHT_TYPE DHT11
 DHT dht(DHT_PIN, DHT_TYPE);


void setup() { Serial.begin(9600); dht.begin(); }


void loop() {
   float h = dht.readHumidity();
   float t = dht.readTemperature();
   if (isnan(h) || isnan(t)) {
     Serial.println("Sensor read failed");
     delay(2000); return;
```

```
  }
  Serial.print("Temperature: "); Serial.print(t);
  Serial.print(" C  Humidity: "); Serial.print(h); Serial.println(" %");
  delay(2000);
}
```

Mini Task 2 Temperature Warning System

Objective. Classify a measurement into operating zones.



Components

- DHT sensor
- Green, yellow, and red LEDs
- Three 220 ohm resistors

Connections

- Connect LEDs to pins 8, 9, and 10 through individual resistors.
- Keep the DHT connection from Mini Task 1.

Procedure

29. Define two temperature boundaries.
30. Turn on green below the first boundary, yellow between boundaries, and red above the second.
31. Ensure other LEDs are turned off in every state.
32. Test by gently warming the air near the sensor without touching or overheating it.

Expected result. One LED indicates the current temperature zone.

Extension challenge. Add a buzzer and require several consecutive high readings before sounding the alarm.

Session 5 Distance and Motion Sensors

Learning outcomes

- Explain time-of-flight distance measurement.
- Convert echo duration to distance.
- Use a PIR sensor as a digital event source.
- Build proximity and motion warnings.

Technical explanation

Ultrasonic time of flight

The HC-SR04 emits an ultrasonic burst after a short trigger pulse. Its ECHO pin remains HIGH for the sound round trip. Distance equals time multiplied by the speed of sound and divided by two. At about 20 degrees Celsius, a convenient conversion is distance in centimeters equals echo microseconds multiplied by 0.0343 divided by two.

Measurement limitations

Soft, angled, narrow, or moving targets can weaken or redirect the reflection. Very close or distant objects may fall outside the practical range. Temperature affects sound speed. Several samples and a median filter can improve stability.

PIR motion sensing

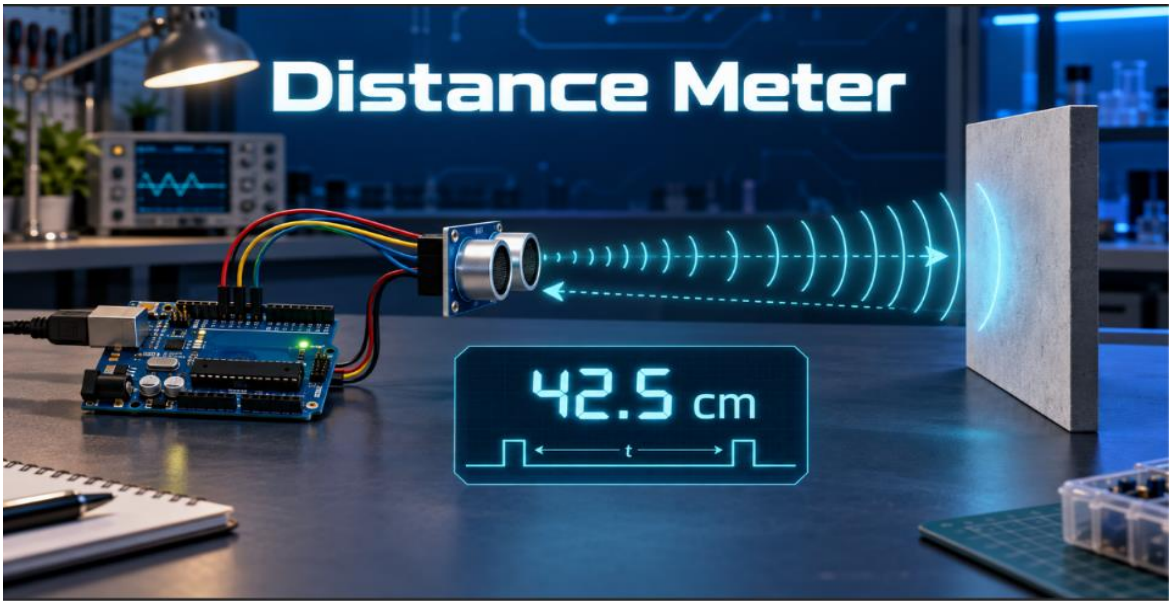
A PIR module detects changes in infrared radiation across sensing zones. It does not measure distance or identify a person. Modules often require a warm-up period and may include sensitivity and hold-time adjustments.

Event versus continuous data

Distance is a continuous measurement, while the PIR module provides a digital motion state. The software architecture should reflect this difference: distance may be filtered and compared with zones, while motion may start a timer or trigger a state transition.

Mini Task 1 Distance Meter

Objective. Measure the distance to a target.



Components

- HC-SR04 ultrasonic sensor

Connections

- VCC to 5 V, GND to GND, TRIG to pin 6, ECHO to pin 7.

Procedure

33. Generate a 10 microsecond trigger pulse.
34. Measure the ECHO HIGH duration with pulseIn and a timeout.
35. Convert time to centimeters.
36. Compare measurements with a ruler at several distances.

Expected result. Distance readings are displayed in centimeters.

```
const int TRIG_PIN = 6, ECHO_PIN = 7;


void setup() {
   Serial.begin(9600);
   pinMode(TRIG_PIN, OUTPUT); pinMode(ECHO_PIN, INPUT);
 }


void loop() {
   digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
   digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
   digitalWrite(TRIG_PIN, LOW);
   unsigned long duration = pulseIn(ECHO_PIN, HIGH, 30000);
   if (duration == 0) Serial.println("No echo");
```

```
  else {
    float distanceCm = duration * 0.0343 / 2.0;
    Serial.print(distanceCm); Serial.println(" cm");
  }
  delay(100);
}
```

Mini Task 2 Parking Assistant

Objective. Convert distance zones into visual and audible feedback.



Components

- HC-SR04
- Three LEDs with resistors
- Buzzer

Connections

- Retain the ultrasonic wiring.
- Connect LEDs to pins 8, 9, and 10; connect an appropriate low-current buzzer to pin 11 and GND.

Procedure

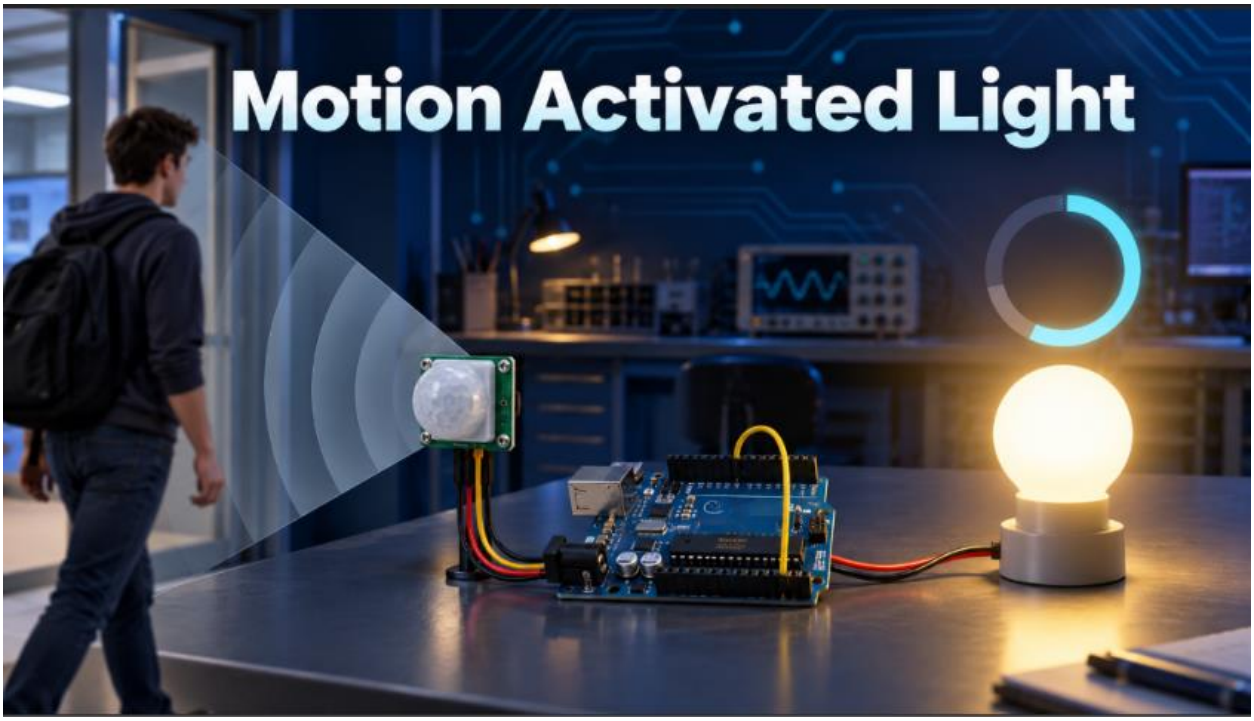
37. Define far, caution, and stop zones.
38. Select the LED for the current zone.
39. Reduce the interval between beeps as distance decreases.
40. Treat a missing echo as an invalid measurement, not zero distance.

Expected result. Warnings become more urgent as the target approaches.

Extension challenge. Collect five readings, sort them, and use the median to reject occasional outliers.

Mini Task 3 Motion Activated Light

Objective. Use a PIR event to control a timed output.



Components

- PIR sensor
- LED and resistor

Connections

- PIR OUT to pin 2, VCC and GND according to module label.
- LED to pin 8 through a resistor.

Procedure

41. Allow the PIR sensor to stabilize after power-up.
42. Turn on the LED when motion is detected.
43. Keep it on for a defined period after the most recent detection using millis.

Expected result. Movement activates the light; it turns off after the timeout.

Extension challenge. Add an armed mode button and a buzzer to create a basic security system.

Session 6 Actuators and Sensor Integration

Learning outcomes

- Differentiate sensors from actuators.
- Control servo position safely.
- Translate sensor data into motion.
- Design a non-blocking automatic gate state machine.

Technical explanation

Sensors and actuators

A sensor converts a physical condition into data. An actuator converts an electrical command into physical action. A complete control system measures the environment, makes a decision, and drives an actuator.

Servo control

A hobby servo receives repeated control pulses representing a target angle. The Servo library generates these pulses. The commanded angle is a position request, not proof of actual position. Mechanical limits and load must be respected.

Power integrity

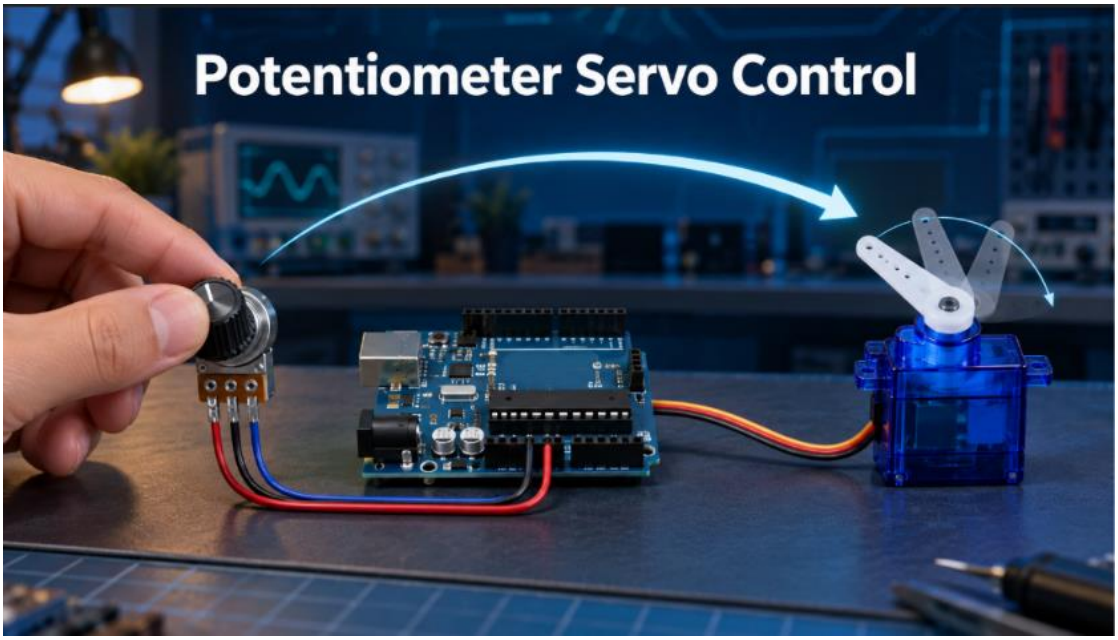
A small unloaded servo may appear to work from the Arduino 5 V pin, but current spikes can reset the board or damage connections. For reliable prototypes, use a suitable external 5 V supply and connect its ground to Arduino ground. Never connect independent supply voltages together unless the design requires it.

State machines

An automatic gate has states such as CLOSED, OPENING, OPEN, and CLOSING. A non-blocking state machine uses sensor conditions and timestamps instead of long delay calls, so the system can continue measuring and respond safely.

Mini Task 1 Potentiometer Servo Control

Objective. Map a human input to actuator position.



Components

- Servo motor
- 10 kohm potentiometer
- Suitable 5 V supply when required

Connections

- Potentiometer wiper to A0 and outer pins to 5 V and GND.
- Servo signal to pin 9; servo supply to appropriate 5 V source; all grounds common.

Procedure

44. Attach the servo in setup.
45. Read the potentiometer.
46. Map 0 to 1023 into a safe angle range such as 10 to 170 degrees.
47. Write the target angle and observe motion.

Expected result. The servo follows potentiometer position.

Extension challenge. Add software limits that can be adjusted through the Serial Monitor.

```
#include <Servo.h>
 Servo gateServo;


void setup() { gateServo.attach(9); }


void loop() {
   int raw = analogRead(A0);
   int angle = map(raw, 0, 1023, 10, 170);
   gateServo.write(angle);
```

```
  delay(15);
}
```

Mini Task 2 Automatic Gate

Objective. Use distance to control a servo gate.



Components

- HC-SR04
- Servo motor
- External supply if required

Connections

- Use the previous HC-SR04 connections.
- Connect servo signal to pin 9 and use a common ground.

Procedure

48. Measure distance.
49. Open the gate when a valid target is closer than the chosen approach distance.
50. Record the time at which the gate opened.
51. Close only after the target is clear and the hold time has expired.
52. Avoid rapid open-close movement by using separate open and close thresholds.

Expected result. The gate opens for an approaching object and closes safely after the area is clear.

Extension challenge. Implement CLOSED, OPEN, and WAITING_TO_CLOSE states without delay.

Session 7 Displays and Data Visualization

Learning outcomes

- Explain the purpose of a human-machine interface.
- Describe I2C addressing and shared bus lines.
- Display formatted sensor values.
- Reduce display flicker and organize multiple screens.

Technical explanation

Why a display matters

A display exposes sensor values, operating modes, warnings, and errors without requiring a computer. A useful interface communicates meaning, units, and system state rather than showing unexplained numbers.

I2C communication

I2C uses SDA for data and SCL for the clock. Devices share the bus and are selected by address. On an Uno, SDA is associated with A4 and SCL with A5. Many LCD backpacks use address 0x27 or 0x3F, but an address scanner should be used when uncertain.

Formatting and refresh

Fixed-width fields prevent old digits from remaining when a new value has fewer characters. The display should not be cleared continuously because that causes visible flicker. Update only changed fields or overwrite complete lines at a reasonable rate.

Screen design

Prioritize critical information. A two-line display might show temperature and humidity normally, then temporarily replace the second line with a warning. Multiple pages can be selected with a button, but the current page and units must remain clear.

Mini Task 1 Environmental Display

Objective. Present sensor data without a computer.



Components

- I2C 16 by 2 LCD
- DHT sensor

Connections

- LCD VCC and GND to the correct supply; SDA to A4 and SCL to A5 on Uno.
- Retain the DHT connection from Session 4.

Procedure

53. Find the LCD address with an I2C scanner if necessary.
54. Initialize the display and backlight.
55. Read validated temperature and humidity.
56. Print labels, values, and units; refresh about every two seconds.

Expected result. The display shows stable temperature and humidity readings.

```
#include <Wire.h>
 #include <LiquidCrystal_I2C.h>
 #include <DHT.h>
 LiquidCrystal_I2C lcd(0x27, 16, 2);
 DHT dht(4, DHT11);

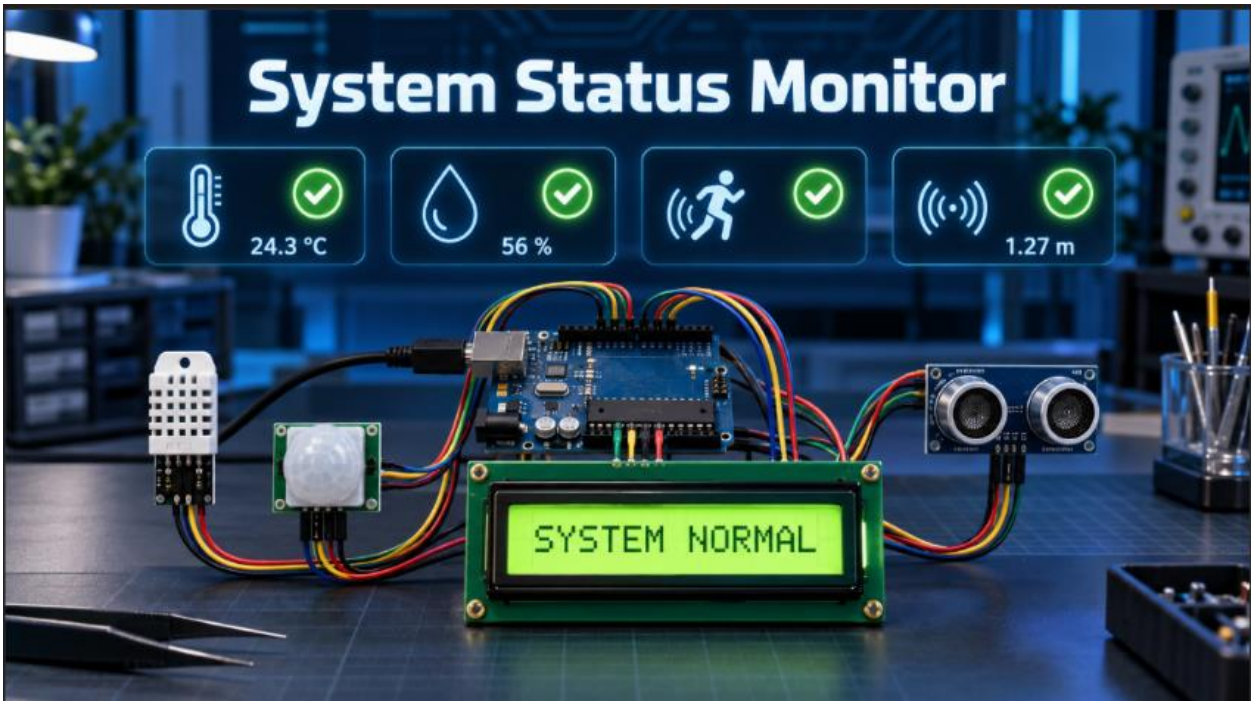

void setup() {
   lcd.init(); lcd.backlight(); dht.begin();
 }


void loop() {
   float t=dht.readTemperature(), h=dht.readHumidity();
   if (!isnan(t) && !isnan(h)) {
     lcd.setCursor(0,0); lcd.print("Temp: "); lcd.print(t,1); lcd.print(" C  ");
     lcd.setCursor(0,1); lcd.print("Hum:  "); lcd.print(h,1); lcd.print(" %  ");
   } else {
     lcd.setCursor(0,0); lcd.print("Sensor error    ");
   }
```

```
  delay(2000);
}
```

Mini Task 2 System Status Monitor

Objective. Convert system data into concise user messages.



Components

- LCD
- One or more previous sensors
- Push button

Connections

- Connect the page button to pin 2 and GND using INPUT_PULLUP.
- Keep I2C and sensor connections unchanged.

Procedure

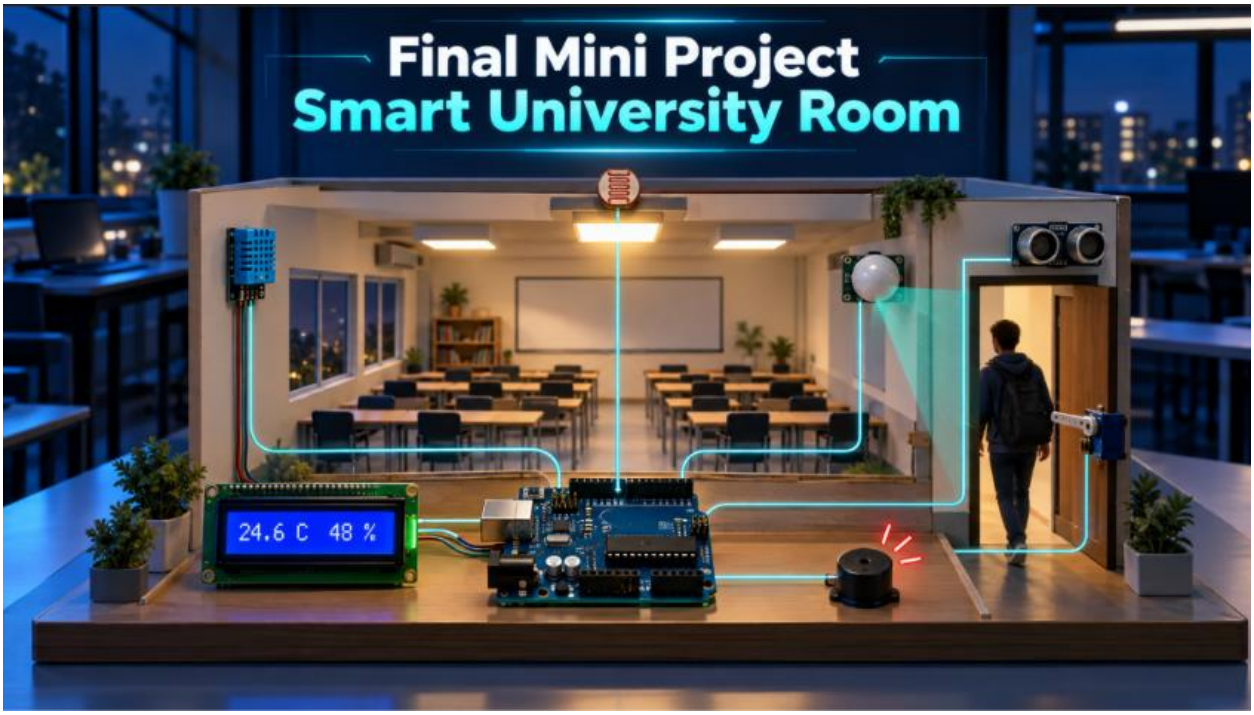
57. Create pages for environmental data, distance, and motion state.
58. Use edge detection to change pages.
59. Display a clear warning when a threshold is exceeded.
60. Avoid clearing the LCD on every loop pass.

Expected result. The button cycles through readable status screens.

Extension challenge. Add an automatic return to the main screen after ten seconds of inactivity.

Final Mini Project Smart University Room

Student teams will integrate the workshop concepts into a room-monitoring and access-control prototype. The project should demonstrate purposeful sensing and control, not simply connect every component. Teams may simplify the hardware if they can explain the design choice and preserve the main engineering functions.



Required functions

61. Turn the room light on when ambient light falls below a calibrated threshold.
62. Detect motion and record the most recent detection time.
63. Measure temperature and humidity at an appropriate interval.
64. Activate a warning when temperature exceeds a defined limit.
65. Open a servo gate when a valid target approaches and close it after the area is clear.
66. Display measurements, operating state, and sensor errors.

Recommended software architecture

Module	Responsibility	Suggested update rate
Light	Read LDR, filter data, apply hysteresis	50 to 200 ms
Environment	Read and validate DHT data	2 s
Distance	Trigger, time, validate, and filter echo	50 to 150 ms
Motion	Read PIR and update last-motion time	Every loop

Gate	Manage servo state and timing	Every loop
Interface	Refresh changed display fields	250 ms to 2 s

Development sequence

67. Write a requirement list and define each input, output, threshold, and safe state.
68. Draw the circuit and create a pin-assignment table before wiring.
69. Test each sensor and actuator separately with a minimal sketch.
70. Create functions for reading sensors and updating outputs.
71. Integrate one module at a time while keeping previous tests working.
72. Replace long delay calls with millis-based scheduling where simultaneous behavior is required.
73. Test normal operation, boundary values, disconnected sensors, missing echoes, and rapid repeated events.
74. Document results, limitations, and improvement opportunities.

Suggested pin map

Device	Pin	Notes
PIR	D2	Digital input with module output
DHT	D4	Digital data
HC-SR04 TRIG	D6	Digital output
HC-SR04 ECHO	D7	Digital input
Room LED	D8	Use resistor
Servo signal	D9	External supply may be required
Buzzer	D10	Check current requirement
LDR divider	A0	Analog input
LCD	A4 and A5	I2C SDA and SCL